

COMP101

Introduction to Programming

2025-26

Lecture-17_3

Iteration Python

nested for loops

Nested loops: overview

- A loop (for loop or while loop) can contain other loops within its loop body

i.e.

- for loop with another for loop in it
- for loop with a while loop in it
- while loop with another while loop in it
- while loop with a for loop in it
- They can also contain selection statements

Nested loops: overview

- A 'nested loop' has another loop or 'if' in it
- 'Outer loop' is the loop which has another loop or 'if' in its body
- 'Inner (nested) loop' is the loop that is in the body of the outer loop
- Outer loop controls the number of iterations of the inner loop
 - Inner loop executes once for each iteration of the outer loop

Nested loops: overview

outer loop start (loop header)

outer loop statements (may not be any)

inner loop start (loop header)

inner loop statements (must have some)

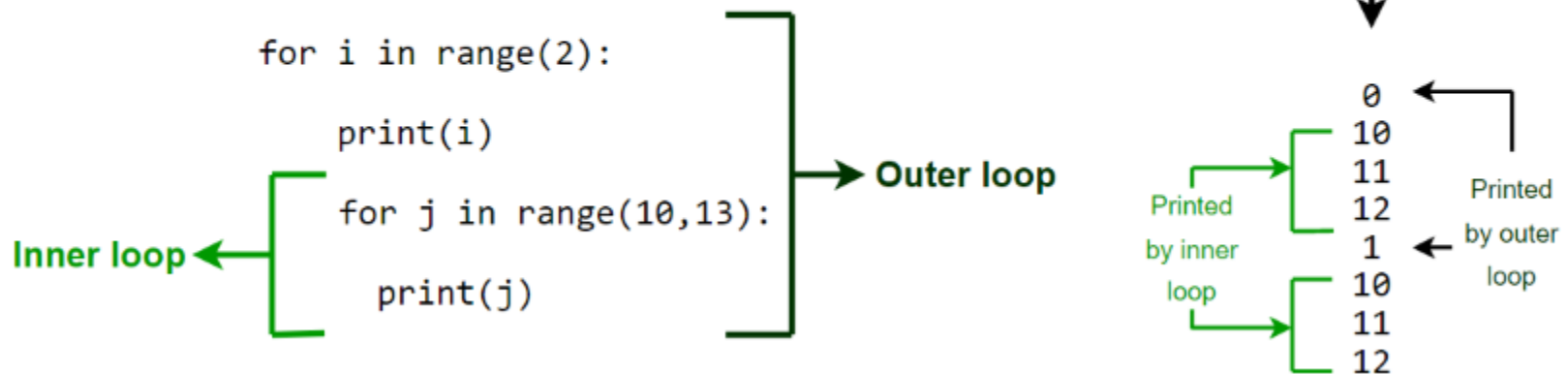
inner loop ends with un-indented line

outer loop ends with un-indented line

Note: indentation determines when outer loop starts and ends and when inner loop starts and ends

Nested for: structure

Python Nested Loop



`for i in range(0,2)` - outer loop must iterate two times for index [0] and [1]

first iteration prints index[0] then execute inner loop

`for j in range(10,13)` – must iterate three times for indices [10], [11] and [12]

first iteration prints index[10], second iteration prints index[11], third iteration prints index[12]

Inner loop done - return to outer loop:

`for i in range(0,2)` outer loop iterates a second time – this time for index[1]

second iteration prints index[1] then execute inner loop

`for j in range(10,13)` – must iterate three times for indices [10], [11] and [12]

first iteration prints index[10], second iteration prints index[11], third iteration prints index[12]

Nested for: problem statement

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30

Requirements:

1. Produce '2-times' and '3-times' tables
2. o/p: multiplier (2 and 3) * multiplicand (1 to 10) = product

Initial design:

Deterministic loop (for loop) to iterate two times – once for '2-times table' then once for '3-times table';

For each loop above, outputs ten lines for multiplicand 1 to 10:

Deterministic loop (for loop) to iterate ten times

Analysis:

The first loop (outer loop) will iterate 2-times;

When this outer loop iterates the first time (for the '2-times table') it must have another loop in it (inner loop) that iterates ten times to print the 2-times table

When this outer loop iterates the second (and last) time (for the '3-times table') it must have another loop in it (inner loop) that iterates ten times to print the 3-times table

Nested for: problem statement

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30

Mechanics:

1. first loop has control and iterates with multiplier = 2
2. second loop iterates ten times to output table for multiplier = 2
3. first loop regains control and iterates with multiplier = 3
4. second loop iterates ten times to output table for multiplier = 3
5. Program ends

Nested for: problem statement

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30

Mechanics:

1. first loop has control and iterates with multiplier = 2
2. second loop iterates ten times to output table with value = 2
3. first loop regains control and iterates with multiplier = 3
4. second loop iterates ten times to output table for value = 3
5. Program ends

pseudocode level 0:

1. Initialise
 - 1.1 set first_value = 2
 - 1.2 set second_value = 3
2. loop outer first_value, second_value
 - 2.1 loop inner 10 times
 - 2.1.1 print times-table
 - 2.2 end loop inner
 - 2.3 end loop outer

Nested for: problem statement

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20

Design: pseudocode level 0:

1. Initialise
 - 1.1 set first_value = 2
 - 1.2 set second_value = 3
2. loop outer first_value, second_value
 - 2.1 loop inner 10 times
 - 2.1.1 print times-table
 - 2.2 end loop inner
 - 2.3 end loop outer

Pseudocode defines the code logic/structure.

Last line of code below -
'print()' - not needed in pseudocode – it is for o/p cosmetics so no effect on the logic

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30

```
for i in range (2,4):  
    for j in range (1,11):  
        print(i,"*",j,"=",i*j)  
    print()
```

#outer loop for multiplier (2 and 3)

#inner loop for multiplicand (1-10)

#inner loop statements

#blank line between tables

Don't use the same var name in both loops

Outer loop does not have to have statements unless needed

Nested for: problem statement

Information only

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20
```

```
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

```
for i in range (2, 4):           #outer loop (2 and 3)
    for j in range (1, 11):      #inner loop (1 to 10)
        print( i, '*', j, '=', i * j )    #inner loop statements clearer?
    print()                      #blank line between tables
```

Analysis of above:

Ranges conform to normal Python start-end syntax, range statement resolves 0 default issue, o/p meets normal expectations.

But -only works with 2 and 3, range statements may confuse

Refinement: start with 1-times table and allow as many as wanted

```
num_of_tables = int(input("How many tables to print? "))
for i in range (1,num_of_tables + 1):    # '+1' to get last number in range (3)
    for j in range (1,11):               # actual range = 1 to 10
        print( i, '*', j, '=', i * j )   # spaces/apostrophes for clarity?
    print()                              #outer loop
```

Nested for: problem statement

Information only

How many tables do you want to print? 2

Printing the 1 times table

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10
```

Printing the 2 times table

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20
```

```
num_of_tables = int(input("Number of tables to print?"))
for i in range (1,num_of_tables + 1):
    for j in range (1,11):
        print( i, '*', j, '=', i * j )
    print()
```

Analysis of above:

Range statement more readable – resolve 0 default, o/p meets expectations

Can expand it usability:

Refinement: print multiple tables and as many multiplicands as wanted

```
num_of_tables = int(input("How many tables to print? "))
num_of_mults = int(input("How many multiplicands? "))
for i in range (1,num_of_tables+1):
    print ("Printing the",i,"times table") #note indentation – outer loop

    for j in range (1,num_of_mults+1): #+1 to get end of range
        print( i, '*', j, '=', i * j )
    print()
```

```
print ('Tables complete')
```

#keep user informed what's happening

Nested for: problem statement

Information only

How many tables do you want to print? 2

How many multiplicands? 5

Printing the 1 times table

1 * 1 = 1

1 * 2 = 2

1 * 3 = 3

1 * 4 = 4

1 * 5 = 5

Printing the 2 times table

2 * 1 = 2

2 * 2 = 4

2 * 3 = 6

2 * 4 = 8

2 * 5 = 10

```
num_of_tables = int(input("How many tables print? "))
num_of_mults = int(input("How many multiplicands? "))
for i in range (1,num_of_tables+1):           #outer loop – num_of_tables
    print ("Printing the",i,"times table")
    for j in range (1,num_of_mults+1):         #inner loop – num-of_mults
        print( i,'*', j, '=' , l * j )
    print()                                   #no further statements – lack of space
```

Analysis:

User has control over number of tables and number of multiplicands

Can alter user dynamics by moving second line to above the inner loop - cosmetic

```
num_of_tables = int(input("How many tables to print? "))
for i in range (1,num_of_tables + 1):
    print ("Printing the",i,"times table")
    num_of_mults = int(input("How many multiplicands? "))
    for j in range (1,num_of_mults + 1):
        print( i, '*', j, '=' , l * j )
    print()
```

Nested for: problem statement

Information only

How many tables do you want to print? 2

Printing the 1 times table

How many multiplicands? 5

1 * 1 = 1

1 * 2 = 2

1 * 3 = 3

1 * 4 = 4

1 * 5 = 5

Printing the 2 times table

How many multiplicands? 4

2 * 1 = 2

2 * 2 = 4

2 * 3 = 6

2 * 4 = 8

#numbers for reference only

```
1 num_of_tables = int(input("How many tables to print? "))
2 for i in range (1,num_of_tables+1):
3     print ("Printing the",i,"times table")
4     num_of_mults = int(input("How many multiplicands? "))
5     for j in range (1,num_of_mults+1):
6         print( i, '*', j, '=', i * j )
7         print()
8 print ('Tables complete')
```

Program is more flexible to user needs:

Maybe can see how the nested loop works here?

line#2 start outer loop

line#3, #4 indented inside the outer loop – they 'belong' to it

line#5 start inner loop - indented inside the outer loop – it 'belongs' to it

line#6, #7 indented inside inner loop – they 'belong' to the inner loop

Nested for: with inner for

- Assign grades to every student (200 students)
 - Every student submits 7 assignments
 - Non-submission is awarded a zero grade
- We know how many students there are
 - Use a deterministic loop (for) – loop for 200 times
- We know how many assignments there are
 - Use a deterministic loop (for) – loop for 7 times

Nested for: with inner for

```
students = int(input("How many students: ")) 3
```

```
assignments = int(input("How many assignments per student: ")) 7
```

```
for i in range (1,students+1):           #outer loop – 3 times
    for j in range (1,assignments+1):     #inner loop – 7 times for each of 3 students
        grade = int(input("Enter grade for student: "))
```

Outer loop does not have to have any statements;

Inner loop must have some indented statements or nothing will happen!

Outer loop iterates first time	then inner loop iterates 7 times
Outer loop iterates a second time	then inner loop iterates 7 times
Outer loop iterates a third time	then inner loop iterates 7 times

Nested for: with inner for

How many students: 3

How many assignments per student: 7

Enter grade for student: 25

Enter grade for student: 35

Enter grade for student: 37

Enter grade for student: 45

Enter grade for student: 68

Enter grade for student: 10

Enter grade for student: 76

Enter grade for student: 89

Enter grade for student: 46

Enter grade for student: 34

Enter grade for student: 70

Enter grade for student: 23

Enter grade for student: 57

Enter grade for student: 67

Enter grade for student: 65

Enter grade for student: 45

Enter grade for student: 34

Enter grade for student: 78

Enter grade for student: 67

Enter grade for student: 23

Enter grade for student: 46

It works:

There are 21 grades entered

3 iterations of outer loop, each
iteration followed by followed by 7
iterations of inner loop

$3 \times 7 = 21$

i.e., we expect 21 lines of output

... what do we think of the output!

Nested for: with inner while

- All 200 students can optionally submit a piece of work for 'formative assessment'- there is no penalty if nothing submitted
 - We know how many students there are (for)
- Assign a grade for every student who submits else assign a zero for nothing needed marking
- We don't know how many submissions (while)

Nested for: with inner while

```
students = int(input("How many students: "))

for i in range (1,students+1):    #outer loop handles students
    print ("This is student",i)
    submission = input("Is there a submission for this student y/n: ")

    while submission == 'y':      #inner loop handles each student grade
        grade = int(input("Enter grade for student:"))
        submission = input("Is there a submission for this student y/n: ")
        #remember to check the control var again

    print ('All processed')      #end of outer loop
```

for with inner while:

Outer for loop must have at least one executable statement in it to initialise the sentry for the inner while loop, i.e. we can't immediately execute the inner while loop straight after the outer for loop

Nested for: with inner if

#identify odd and even numbers – o/p is 0 to 5

```
for i in range(6):    #outer loop with range+1 to o/p 5
```

```
    print (i)
```

```
    if i%2 == 0:      #nested 'if' uses modulo to give remainder
```

```
        print(i,"is an even number")
```

```
    else:             #else 'belongs' to the if note indentation
```

```
        print (i,"is an odd number")
```

```
    print()           #belongs to the 'for' – note indentation
```

0
0 is an even number

1
1 is an odd number

2
2 is an even number

3
3 is an odd number

4
4 is an even number

5
5 is an odd number

If statement has normal structure as with any if statement – its just nested (indented) under the for

for – break: Loop optimisation

#poisonous fungi in the list – don't eat it

```
partyFoodList = ["PRINCE", "DECEIVER", "LILAC", "PUFF"]
```

```
mushroom = ""
```

```
for mushroom in partyFoodList:
```

```
    mushroom = input ("What mushroom did you eat? (prince, deceiver, lilac or puff): ")
```

```
    mushroom = mushroom.upper()
```

```
    if mushroom == "LILAC":
```

```
        break
```

```
    else:
```

```
        print (mushroom,"is okay")
```

```
print ("Ambulance is on the way")
```

'break' halts loop execution early:
We 'break out' of the loop and it transfers control to the first un-indented statement after the loop construct. It reduces execution time.

Can become complex - good documentation is needed to avoid confusion

for – continue: Loop optimisation

#output only positive numbers

```
numList = [1, 2, 3, -4, 5, 6, 7, 8, 9, 10]
```

```
number = 0
```

```
for number in numList:
    if number < 0:
        continue
    else:
        print (number,"",end="")
```

```
print ("Loop completed")
```

```
1 2 3 5 6 7 8 9 10 Loop completed
```

‘continue’ – ‘skips over’ an item in the list. Here, it ignores negative numbers.

It halts current execution, but remains in the loop because it returns to the test condition.

It doesn’t do the ‘else’ part

Execution remains in the loop until the loop ends normally. It doesn’t terminate the loop prematurely

Can become complex - good documentation is needed to avoid confusion

Nested for: for-else

Information only

#a list is invalid if there are any negative numbers in it

for i in (1, 2, 3, 4, 5, 6, 7, -8, 9, 10): #can't use for i in range as -ve int will not be included

 print(i)

 if i <=0:

 print (i,"is -ve")

 else: #belongs to the nested if

 print (i,"is +ve")

else: #optional and belongs to the outer for loop

 print ("loop-else executed - all numbers processed")

All numbers in the list output with a message 'positive or negative number'

o/p examined visually by operator for negative numbers where which invalidate the list

Technically the else that belongs to the outer loop is redundant – code works okay without it.

Think about design before using for...else. It only executes when the loop executes normally

```
1
1 is +ve
2
2 is +ve
3
3 is +ve
4
4 is +ve
5
5 is +ve
6
6 is +ve
7
7 is +ve
-8
-8 is -ve
9
9 is +ve
10
10 is +ve
```

Nested for: for-else

Information only

10,000 numbers – any negative number makes the list invalid:

A visual inspection of 10,000 numbers is unrealistic. Stop the loop if a negative number is found

No need to process any further numbers once a negative number is i/p

'break' will exit the loop prematurely when negative number found and the for-else never executes

for-else used is usually used with a break statement in loop body – side-effects occur when a for-else does not associate with a break

```
for i in range (1,10000):      #any -ve number in the list makes list invalid
    if i <=0:
        print (i,"is -ve")
        print ("-ve number found")
        break                #stop and exit the outer loop early when -ve number i/p
    else:                      #else belongs to if
        print (i,"is +ve")
else:                          #else belongs to for
    print ("loop-else executed - all numbers processsed – no negative numbers")
```

Problem: for-else used without break

Information only

- Side-effects can result in loops using else statement without a break statement
- i/p integer: if int = 1 to 7 then o/p 'number is a winner' else o/p 'Not a winner'
- Side effect: both messages are o/p because when loop ends (at end of range) the else clause executes

```
myNumber = int(input("Enter your secret number ")) 7
```

```
for i in range(11):          #i/p range 1 to 10: Can use for i in range(1, myNumber + 1)
```

```
    if i == myNumber:
```

```
        print ("Your number is a winner")
```

```
else:                          #belongs to the outer for loop
```

```
    print ("Not a winning number")
```

```
Enter your secret number 7
```

```
Your number is a winner
```

```
Not a winning number
```

Side-effect: else always executes when no break statement is encountered:

There is no break statement in the loop so the for-else always executes and we o/p both messages at end of loop

for else

<https://www.tutorialspoint.com/How-to-use-else-statement-with-Loops-in-Python>

- includes while loop

For – pass: Used for development only

- pass can be used as a ‘placeholder’
 - It doesn’t do anything but allows us to check the progress of the code for taking further action
- Useful in loops, functions and if statements
 - Prevents an error when we don’t want any code to be executed yet

pass

Information only

#used in a loop

```
for i in [1,2,3]:
```

```
    pass    #eventually replace this with active code
```

#used in selection

```
a = 1
```

```
b = 3
```

```
if a > b:
```

```
    pass    #eventually replace this with active code here
```

‘pass’ does not do anything

‘pass’ allows execution to continue without ‘throwing an error’

Used as a ‘placeholder’ that needs replacing with active code

- Without ‘pass’ we get an error as active code is expected
 - Active code is expected after the colon
- Replace ‘pass’ with active code eventually

Max levels of nesting

- Python recognises 20 levels of nesting
- Check design if using lot of nesting
 - code/logic may need amending
- Be aware of using break, continue, for-else
 - Useful in specific scenarios
 - Can cause side-effects if not used properly